Measuring the Defenders, Held Out: A Spec-Versioned, Framework-Mapped Coverage Benchmark for Model Context Protocol Security Proxies

Anonymous Author(s)
Submission under double-blind review

***Abstract*—The Model Context Protocol (MCP) connects LLM agents to tools, and a dense 2025–2026 literature documents its attack surface. Existing MCP security benchmarks measure how well an *agent* resists attacks; none measure how much of that surface a *defensive proxy or gateway* covers, and none map results to the governance frameworks organizations use. We present (1) a threat–control crosswalk of 32 MCP attack vectors—24 pre-existing and 8 introduced by the breaking 2026-07-28 protocol revision—mapped to architectural layer, STRIDE, NIST AI RMF, the OWASP LLM and Agentic Top 10s, and the NSA MCP guidance, with a published rating codebook and a blind second-rater pass (primary-layer $\kappa = 0.75$, STRIDE $\kappa = 0.94$); and (2) an open, vendor-neutral *defense-side* benchmark that drives each proxy through its real code against matched malicious/benign fixtures and reports coverage *per protocol revision*. Because our own proxy was developed against the benchmark, we add what a benchmark of this kind must have: a *pre-registered held-out corpus* (48 fixtures written after every defender was frozen, under domain/lexical/surface/encoding shift rules) and a 337-item *benign-only corpus* (256 verbatim tool definitions harvested from public MCP servers plus 81 hard negatives). The reference proxy covers 55% of the 24 pre-existing vectors on the development corpus but 43% held-out—a 13-point generalisation gap that exposes phrase-brittle heuristics—at a benign false-positive rate of 3.9% (95% CI 2.3–6.5%). One competing egress firewall's development coverage disappears entirely held-out. 9 of 32 vectors are covered by no measured tool. We argue the held-out gap, not the development score, is the number a defender should report, and release everything needed to reproduce and extend the measurement.**

***Index Terms*—Model Context Protocol, LLM agents, agentic AI security, security benchmark, held-out evaluation, NIST AI RMF, OWASP, defense-in-depth**

I. Introduction

MCP standardizes how an LLM agent discovers and calls external "tools" exposed by "servers." Its adoption has been accompanied by systematizations, formal taxonomies, attack benchmarks, and attestation proposals (Section II) that establish *what can go wrong*. A practical question is underaddressed: **given a defensive tool in front of MCP servers—a proxy, gateway, or scanner—how much of the known attack surface does it actually cover, and does that coverage survive attacks it was not developed against?**

Answering this well requires a shared taxonomy, a mapping to the governance frameworks security and procurement teams reason in, an executable way to test a defender, and—because the benchmark's authors also build one of the defenders—controls that separate "guided development" from "generalisation." A first version of this work was reviewed at a 2026 security workshop; the expert reviewer's three objections (co-development of benchmark and tool, a small corpus with an over-general "zero false positives" claim, and a single-author framework mapping) shaped everything new here. We contribute:

- **A spec-versioned threat–control crosswalk** (Section III): 32 vectors with an `appliesTo` protocol-revision axis, so that coverage is never quoted revision-free; the 8 2026-07-28 vectors were verified against specification text, which corrected two vendor-sourced claims.
- **A rating codebook and agreement measurement** (Section III-A): a second, blind rating pass and open adjudication of every disagreement.
- **A defense-side benchmark with three corpora** (Section IV): a development corpus, a *pre-registered held-out corpus* authored after defender freeze, and a large benign-only corpus with confidence intervals on the false-positive rate.
- **An empirical evaluation** (Section VI) of three open-source proxies and a null baseline, reporting development and held-out coverage side by side, per revision, with false-positive rates.

Our aim is not to rank products but to map coverage, expose gaps, and provide reusable, honest infrastructure.

II. Related Work

Recent works are preprints whose taxonomies are their authors' constructs; we cite them as *proposed by*, not consensus.

Threat taxonomies. An SoK separates adversarial threats from epistemic hazards across MCP primitives [1]; a formal framework proposes 7 categories and 23 vectors from a large tool corpus [2]; a STRIDE/DREAD model finds tool poisoning the most impactful client-side vulnerability [3]; a defense-placement taxonomy classifies defenses by enforcing layer and finds protection "uneven and predominantly tool-centric" [4]. Our crosswalk reuses these vector families and

adds the framework mapping, the protocol-revision axis, and a reliability measurement none provides.

Attack-side benchmarks. MSB [5] measures LLM-agent resistance with real MCP execution; MCPTox [6] scopes to tool poisoning; AgentDefense-Bench [7] scores a defender's *detection accuracy* over aggregated datasets; MCP-Bench [8] measures capability. None measures a defensive proxy's *coverage* of a mapped surface, and none is spec-versioned. All were authored against the pre-2026-07-28 protocol shape.

The 2026-07-28 revision. The revision removed the `initialize` handshake and protocol-level sessions, replaced server-initiated requests with in-band "MRTR" (`input_required` + opaque `requestState`), mirrored routing metadata into HTTP headers with a normative `-32020` HeaderMismatch error, made list results cacheable (`ttlMs`, `cacheScope`), promoted Tasks to an official extension, and deprecated Roots, Sampling, Logging and DCR [9]. The new surface is overwhelmingly a gateway-layer surface—precisely the unit under test here.

Attestation and provenance. ETDI [10] and the Trustworthy MCP Registry blueprint [11] are complementary; the vectors they address are, we show, structurally outside any content-scanning proxy's reach.

Scanners and gateways. Practitioner tools include MCP-Scan (now a cloud-gated Snyk scanner [12]), the Lasso and EnkryptAI gateways, Docker's isolation gateway, and the proxies evaluated here. We score locally deterministic defenders and treat cloud-model defenders as observable but not reproducibly scoreable.

III. The Spec-Versioned Threat–Control Crosswalk

The crosswalk is both the benchmark's rubric and a standalone, machine-readable artifact (`rubric/crosswalk.json`, CC-BY-4.0). It enumerates 32 vectors, each mapped to architectural layer (host-orchestration, client, transport, server, tool, registry-supply-chain, following [4]), STRIDE class, NIST AI RMF function, NSA MCP Security recommendation area [13], OWASP LLM Top 10 (2025) [14] and OWASP Agentic Top 10 (2026) [15]. Table I lists the 8 vectors added for 2026-07-28; the 24 pre-existing vectors are those of Table IV.

Protocol-revision axis. Every vector carries `appliesTo`: the list of protocol revisions in which its mechanism exists. The scorer reports coverage over the vectors applicable to each revision, and separately over the "new-only" set. A coverage number quoted without a revision is meaningless once the protocol has moved; to our knowledge no other MCP security benchmark models this.

Verification against specification text. The 2026-07-28 vectors were first drafted from vendor analyses and then checked line-by-line against the specification. Two changed materially: the Tasks extension has *no* `tasks/list` (removed deliberately, cited by the spec as a security improvement), so the planned enumeration variant does not exist; and MCP Apps (SEP-1865 [16]) mandates a sandboxed iframe under an egress-blocking CSP, so the vendor framing of an "open XSS surface" overstates it—the testable property is host *conformance*. A third correction separated `requestState` (normative MUSTs) from tool state handles (a non-normative section), which are now distinct, differently weighted fixtures.

TABLE I
Vectors introduced by the 2026-07-28 revision (`appliesTo:[2026-07-28]`).

#	Vector	Layer	Also known as
25	Header/body routing desync (Header-Mismatch)	transport	Mcp-Name mismatch; routing-header confusion
26	MRTR requestState forgery / replay	server	state-handle tampering; cross-principal replay
27	List-result cache poisoning / stale-cache defense suppression	client	ttlMs pinning; cacheScope confusion
28	MRTR in-band input phishing / sampling injection	client	credential elicitation; input_required phishing
29	Task authorization bypass	server	guessable task IDs; cross-principal tasks/get
30	MCP Apps sandbox non-conformance	host-orchestration	ui:// resource egress; host iframe sandbox
31	Roots deprecation: no enforced filesystem boundary	host-orchestration	missing roots; path scope gap
32	Transport / revision downgrade	transport	legacy session acceptance; HTTP+SSE fallback

TABLE II
Inter-rater agreement, Rater A (primary) vs. Rater B (blind), 24 vectors, pre-adjudication.

Dimension	Primary κ	Mean Jaccard	Pooled κ	Exact
mcpLayer	0.75	0.64	0.60	10/24
stride	0.94	0.85	0.84	17/24
nistAiRmf	n/a	0.74	0.62	11/24
owaspLlm2025	n/a	0.73	0.76	14/24
owaspAgentic2026	n/a	0.69	0.71	13/24
nsaGuidance	n/a	0.83	0.74	17/24

A. Mapping reliability

Framework mappings are judgments. To make them reviewable we publish a *rating codebook* with a decision rule per dimension, a rating sheet containing only vector id, name, aliases and description, and an agreement tool. A second rater (Rater B)—a large-language-model rater following the codebook and blind to the primary mapping—rated the 24 pre-existing vectors. Table II reports Cohen's κ on primary labels (layer, STRIDE), mean per-vector Jaccard over the full label sets, and pooled per-label κ; interpretation follows [17]. Agreement is substantial (primary-layer $\kappa = 0.75$, STRIDE $\kappa = 0.94$; Jaccard 0.64–0.85). The 62 set-level disagreements were adjudicated in the open; 12 mappings changed and one codebook rule was clarified. We report the *pre*-adjudication figures as the headline, since post-adjudication agreement is inflated by construction.

We are explicit about the limit: Rater B is not an independent human expert. It demonstrates that the codebook is followable by a rater who did not write the crosswalk and it makes every disagreement public. A human third-rater slot is open in the artifact, and the 8 new vectors remain single-rater and are labelled as such.

IV. Benchmark Methodology

A. Design

Each vector has one or more *test cases*: a static JSON object with a *malicious fixture* (the attack as data—a tool definition, call, result, registry entry, HTTP request, list result, or scenario), a near-identical *benign control*, the expected defender behavior, and a provenance note. Fixtures are static data, never live exploits; hosts are reserved names.

B. Adapters and the runtime-integrity rule

Each tool provides a thin adapter exposing `assess(input)→{detect, enforce}` that drives the tool's real code (imported detection functions, or its CLI/SDK with a committed configuration). One rule keeps scores honest: **report only what the tool actually inspects at runtime—never what its rules could match if pointed at data the tool never sees.** An unwired check is a miss, however good the function is in isolation.

C. Scoring

Per test case: not flagged → **none**; flagged but the benign control is also flagged → **none** and a **false positive**; flagged with a clean control → **detect** (warn) or **enforce** (block). Weights are enforce $= 1$, detect $= 0.5$. A vector with several fixtures publishes *Capability* ($\max_i w_i$, best case), *CorpusRobustCoverage* ($\text{mean}_i w_i$, the headline) and *Guaranteed* ($\min_i w_i$). The mean, not the max, is the headline because an independent reviewer of the public artifact showed that max-aggregation let a tool that missed an evasion variant keep full credit; a false positive on any variant zeroes that fixture and drags the vector's mean down. The headline is a descriptive coverage measure over a fixed, non-exhaustive corpus—*not* a procurement or substitutability ranking.

D. Three corpora

Development corpus (51 cases, 32 vectors). Used to find and fix gaps in the reference proxy over the study. Coverage on it demonstrates that the benchmark *guides* development; it is a weak generalisation claim.

Held-out corpus (48 cases, 24 vectors). Governed by a protocol registered *before* any fixture was written: (1) every defender is frozen at a recorded version/commit; (2) fixtures are authored from vector descriptions and the attack literature only, without consulting any detector's rules or tests; (3) each fixture belongs to a named transformation family—**D**omain shift (no development tool or domain reused), **L**exical shift (no development trigger phrasing, hosts or example secrets; enforced by the generator), **S**urface shift (the vector expressed through a different fixture type), or **E**ncoding shift (a realistic obfuscation absent from development); (4) every fixture has a matched control; (5) ≥ 2 fixtures per vector, at least one D or L; (6) results are published as measured on the first run—a malformed fixture may be removed and logged, never edited to change an outcome; (7) once published, the set retires into the next development corpus and a fresh held-out set is required for any new claim. The family split is D 23 / L 10 / S 5 / E 10.

TABLE III
Headline: CorpusRobustCoverage on the 24 pre-existing vectors (development vs. held-out, gap in points), on the 8 2026-07-28-only vectors, and benign-corpus false positives. Matched-control FP is 0 for every tool on both paired corpora.

Tool	Class	Dev (24)	Held-out (24)	Gap	2026-07-28-only (8)	Benign FP (n=337)
OurProxy	runtime proxy	55%	43%	13	44%	13 (3.9%)
mcp-firewall	runtime proxy	8%	10%	-2	13%	2 (0.6%)
pipelock	egress firewall	6%	0%	6	0%	3 (0.9%)
null baseline	control	0%	0%	0	0%	0 (0.0%)

Benign-only corpus (337 items). Matched controls test discrimination on near-identical pairs; this corpus tests specificity at scale. 256 items are *verbatim* tool definitions harvested by starting 21 public MCP servers exactly as a user would and recording `tools/list` with package/version provenance. 81 are authored hard negatives across every fixture type: legitimate outputs and calls that mention passwords, deletion, shells, base64, the cloud-metadata address, or "ignore my previous message." Every flag is a false positive; we report the rate with a Wilson 95% interval [18], broken down by input kind.

E. Reproducibility rule

A defender is scored only if its detection runs locally and deterministically; cloud-model defenders are observed, not scored.

V. Experimental Setup

We evaluate four adapters. **null-baseline** returns "not detected" for everything. **OurProxy** (runtime proxy, Apache-2.0) is driven by importing its real detection code—definition/text scanning with evasion normalization, cross-tool correlation, cross-server taint, argument and command-injection validation, config-drift and identity checks, inline secret redaction, and, since its 2026-07-28-aware release, header/body coherence enforcement on its HTTP listener (`-32020`) and cache-policy clamping on upstream list results—under its default `balanced` profile. **mcp-firewall** (runtime proxy, Python, AGPL-3.0) is driven through its real SDK with a committed configuration. **pipelock** (egress firewall, Go) is driven through its offline URL/egress and MCP-response scanners. Versions and commits are pinned in the held-out manifest. OurProxy is developed by the authors; it is scored by the same harness alongside the others and the null baseline, which is exactly why the held-out corpus exists. For the 24 pre-existing vectors its detection code was frozen before the held-out protocol was registered; its 2026-07-28 release touched only the two new-vector checks.

VI. Results

Development vs. held-out (Table III, Table IV). On the 24 pre-existing vectors OurProxy covers 55% (13.3/24) of the development corpus and 43% (10.3/24) of the held-out corpus: a 13-point generalisation gap, with zero held-out false positives. The gap is concentrated: 4 vectors covered in development are

TABLE IV

PER-VECTOR COVERAGE. E = ENFORCE, D = DETECT, – = NONE; THE NUMBER IS THE VECTOR'S CORPUSROBUSTCOVERAGE IN PERCENT. Δ IS HELD-OUT MINUS DEVELOPMENT FOR OURPROXY. †VECTORS INTRODUCED BY 2026-07-28 HAVE NO HELD-OUT SET YET (RULE 7).

#	Vector	OurProxy dev	OurProxy held-out	Δ	mcp-firewall / pipelock fw dev	fw h-o	pl dev / h-o
1	Tool poisoning	D 50	– 0	-50	– 0	– 0	– 0 / – 0
2	Tool name collision / shadowing	D 50	D 50	0	– 0	– 0	– 0 / – 0
3	Rug pull / dynamic capability mutation	E 100	E 100	0	– 0	– 0	– 0 / – 0
4	Out-of-scope parameter injection	E 100	E 100	0	– 0	E 100	– 0 / – 0
5	Prompt injection via tool results	D 50	D 25	-25	– 0	– 0	D 33 / – 0
6	Indirect / retrieval injection	E 75	D 50	-25	E 50	– 0	D 25 / – 0
7	Cross-tool data exfiltration / confused deputy	E 100	E 100	0	E 100	E 50	E 50 / – 0
8	Tool-transfer / cross-server chaining	E 50	E 50	0	– 0	– 0	– 0 / – 0
9	False-error escalation	– 0	– 0	0	– 0	– 0	– 0 / – 0
10	Package / name squatting in registry	– 0	– 0	0	– 0	– 0	– 0 / – 0
11	Supply-chain poisoning (unverified provenance)	– 0	– 0	0	– 0	– 0	– 0 / – 0
12	Configuration drift	D 50	D 25	-25	– 0	– 0	– 0 / – 0
13	Sandbox escape	– 0	– 0	0	– 0	– 0	– 0 / – 0
14	Schema / validation bypass	E 100	E 100	0	– 0	– 0	– 0 / – 0
15	Man-in-the-middle (transport)	D 25	– 0	-25	– 0	– 0	– 0 / – 0
16	DNS rebinding (local servers)	E 100	E 50	-50	– 0	– 0	– 0 / – 0
17	Server impersonation / identity spoofing	E 75	E 75	0	– 0	– 0	– 0 / – 0
18	Excessive permission / privilege escalation	D 50	D 50	0	– 0	– 0	– 0 / – 0
19	Credential / token theft via passthrough	E 75	E 50	-25	E 50	E 50	– 0 / – 0
20	Consent fatigue / over-broad grants	– 0	– 0	0	– 0	– 0	– 0 / – 0
21	Command injection in tool execution	E 100	E 100	0	– 0	E 50	– 0 / – 0
22	System-prompt / context leakage via tools	D 25	– 0	-25	– 0	– 0	D 25 / – 0
23	Mid-session tool injection (MSTI)	E 100	E 100	0	– 0	– 0	– 0 / – 0
24	Multi-tool split poisoning (ShareLock)	D 50	– 0	-50	– 0	– 0	– 0 / – 0
25	Header/body routing desync (HeaderMismatch)†	E 100	–	–	– 0	–	– 0 / –
26	MRTR requestState forgery / replay†	– 0	–	–	– 0	–	– 0 / –
27	List-result cache poisoning / stale-cache defense suppression†	E 100	–	–	– 0	–	– 0 / –
28	MRTR in-band input phishing / sampling injection†	– 0	–	–	– 0	–	– 0 / –
29	Task authorization bypass†	E 50	–	–	– 0	–	– 0 / –
30	MCP Apps sandbox non-conformance†	– 0	–	–	– 0	–	– 0 / –
31	Roots deprecation: no enforced filesystem boundary†	E 100	–	–	E 100	–	– 0 / –
32	Transport / revision downgrade†	– 0	–	–	– 0	–	– 0 / –

missed entirely held-out—tool poisoning (both the domain-shifted directive and the HTML-comment variant), multi-tool split poisoning (both a base64-shard and a numbered-fragment staging), transport MITM (a plaintext WebSocket and TLS-verification-off), and system-prompt leakage—while four more degrade. Every lost vector is one defended by a *phrase or pattern heuristic*; every vector that held (rug pull, out-of-scope arguments, schema bypass, command injection, cross-tool exfiltration, mid-session injection) is defended by a *structural* check (hash pinning, schema validation, data-flow tracking). That is the benchmark's most useful finding about the tool, and it would have been invisible without the held-out set. mcp-firewall is flat (8% → 10%): its checks are narrow but not corpus-fitted. pipelock's development coverage (6%) *vanishes* held-out (0%): its injection-scanner matches were specific to the development fixtures' phrasing and its egress scanner never saw a blockable destination in the held-out hosts—an instance of corpus-encoding sensitivity we had previously only argued about.

Per revision. On all 32 vectors OurProxy reaches 52% (16.8/32; Capability 59%; 15 enforced, 8 detected, 9 none). On the 8 2026-07-28-only vectors it reaches 44% (3.5/8): header/body desync (all three fixtures, including the base64-sentinel evasion and the task-routing variant) and list-cache poisoning (all three, including the stale-cache-suppresses-rug-pull composition) are enforced; MRTR forgery and phishing, task authorization, MCP Apps conformance and transport downgrade are honest misses because the proxy does not yet speak those surfaces. mcp-firewall catches one new vector (the roots-scope call, via its file-access rule); pipelock none.

Benign-corpus false positives (Table V). OurProxy flags 13/337 (3.9%, CI 2.3–6.5%). 4/4 of those are the *by-design* cost of trust-on-first-use pinning flagging benign definition updates; we report them in their own row rather than exclude them. On the 256 verbatim third-party definitions it flags 1/256 (a monitoring tool whose description legitimately mentions webhooks). The remaining flags are on authored hard negatives: a search query for the word "password," a path to `.env.example`, an e-mail saying "ignore my earlier note," and result text about password resets, shell history and an example environment file. mcp-firewall flags 2/337 (0.6%) and pipelock 3/337 (0.9%); pipelock's includes a loopback development URL blocked as SSRF. "Zero false positives" on matched controls remains true for all tools on both paired corpora, and we now say what it does and does not show.

Defense in depth. 23 of 32 vectors are covered by at least one tool; 9 by none—the five pre-existing gaps (false-error escalation, package squatting, provenance-gap supply-chain poisoning, sandbox escape, consent fatigue) plus four 2026-07-28 surfaces (MRTR state forgery, MRTR phishing, MCP Apps conformance, transport downgrade). None is addressable by a content-scanning runtime proxy; they need attestation and

TABLE V
FALSE POSITIVES ON THE BENIGN-ONLY CORPUS BY INPUT KIND.

Input kind	n	OurProxy	mcp-firewall	pipelock
tool-definition	256	1	0	0
tool-result	24	5	1	2
tool-call	20	3	1	0
tool-call-sequence	8	0	0	0
multi-server-registry	5	0	0	0
capability-grant	4	0	0	0
definition-change	4	4	0	0
http-request	4	0	0	0
scenario	4	0	0	1
list-result	3	0	0	0
config-snapshot-diff	2	0	0	0
server-identity	2	0	0	0
cache-invalidation	1	0	0	0
Total	337	13 (3.9%, CI 2.3–6.5)	2 (0.6%)	3 (0.9%)

a verified registry, OS isolation, client-side consent controls, and protocol-aware gateways. A proxy is necessary but not sufficient.

Evasion robustness. On the development evasion set, OurProxy catches zero-width/bidi, homoglyph and base64 variants after its normalization stage; the held-out **E** family shows the limit of that stage: HTML-comment and markdown-comment hiding, JSON-LD embedding, newline-separated command injection and a loopback-lookalike Origin were caught (E: 5/10 for OurProxy), while base64 *shards* spread across tools and numbered sentence fragments were not.

VII. Discussion

Guided development is not generalisation. Over the study the benchmark took OurProxy from 9% to 63% Capability on the development corpus by surfacing concrete, layer-specific gaps and verifying each fix. That is what a coverage benchmark is for. But the held-out gap shows how much of that number was the corpus: structural checks generalise; heuristic checks over-fit the phrasing they were built against. A defender that reports only its development score is reporting how well it learned the benchmark.

Corpus-encoding sensitivity, measured. An earlier corpus expressed exfiltration with placeholder text; adding realistic encodings raised one tool from 5% to 13% without changing the tool. The held-out set is the systematic version of that observation, and it cut the other way too: pipelock's coverage was an artefact of the development encodings.

Adapter faithfulness. Matched controls and the benign corpus both guard against adapter over-reach. The benign corpus exposed an adapter bug before publication: the OurProxy adapter reported a cache-policy "clamp" on list results carrying no hints, which the runtime never does (it returns early). Without the benign corpus this would have inflated the new-vector score. A pipelock adapter bug found earlier the same way (synthesizing an egress URL from an inbound host) is documented in the artifact.

By-design false positives. A rug-pull pinner flags every definition change, malicious or not. Whether that is a false positive depends on the deployment; reporting it in its own row lets a reader decide, and hiding it would have been the worse choice.

Why coverage, not a score. Totals move with the corpus; the per-vector, per-revision map is the contribution. mcp-firewall and pipelock each *enforce* on vectors where OurProxy only warns, so a deployment combining classes covers more than any one tool—the practical reading of defense-in-depth the numbers support.

VIII. Limitations and Threats to Validity

- **Held-out fixtures are still author-written.** Blindness to detector source and the enforced family rules are the controls on that; an externally red-teamed set would be stronger and is an open invitation in the artifact.
- **Rater B is an LLM, not an independent human.** The codebook and disagreement log are published so a human rater can be added and reported.
- **Four defenders, one of them ours.** The board is thin; adapters for AWS AgentCore Gateway, MCP-Scan, Docker MCP Gateway and IBM ContextForge are planned. Access-control/isolation gateways and cloud-model defenders are out of scope for scoring.
- **Corpus size.** 51+48 attack fixtures and 337 benign items are larger than before but still small; per-vector numbers rest on 2–5 fixtures.
- **The 2026-07-28 vectors are new and single-rater**, and their fixtures have no held-out set yet (protocol rule 7).
- **Taxonomy is a synthesis, not a standard**; mappings are indicative even where agreement is high.
- **Configuration dependence.** Results depend on each tool's committed policy.

IX. Future Work

Externally authored held-out sets; a human third rater; adapters for the gateways above; EU AI Act and ISO/IEC 42001 crosswalk columns; a decision-indexed "required gap closure" metric for frozen deployment profiles; and an attestation-conformance suite for the registry/provenance vectors no proxy can reach.

X. Conclusion

We introduced a spec-versioned, framework-mapped crosswalk of 32 MCP attack vectors with measured mapping reliability, and a defense-side benchmark that reports a proxy's coverage per protocol revision on development and held-out corpora, with false-positive rates at scale. The reference proxy's 55% development coverage becomes 43% held-out; that gap—and the structural-versus-heuristic pattern behind it—is the result we think defenders should be measured by. 9 of 32 vectors remain undefended by any measured tool, arguing for layered defense across distinct mechanism classes. Everything needed to reproduce, audit, and extend these measurements is released.

Availability

The benchmark (rubric, three corpora, adapters, runner, agreement tooling, results, and the held-out manifest) and the reference proxy are released open-source (rubric/data CC-BY-4.0, code Apache-2.0). For double-blind review the artifact is an anonymised mirror generated by a leak-checking script from a history-free export: https://anonymous.4open.science/r/OurBench. De-anonymised repositories and archival DOIs will be provided at camera-ready. One evaluated tool (OurProxy) is developed by the authors, as disclosed above.

Use of Large Language Models

An LLM assistant was used to format the manuscript, edit prose, generate LaTeX tables from measured results via a script, and—as disclosed in Section III-A—to act as the blind second rater (Rater B) following the published codebook. All benchmark design, fixtures, measurements and conclusions are the authors' own; no experimental result was generated by an LLM, and all citations were verified against primary sources.

Prior Reviews

This paper was previously submitted to a 2026 security workshop and desk-rejected for an artifact-anonymity violation (an author name in the anonymised mirror), with two reviews already filed. The reviews and a point-by-point account of how each was addressed are appended as required by the venue.

References

[1] S. Gaire, S. Gyawali, S. Mishra, S. Niroula, D. Thakur, and U. Yadav, "Systematization of knowledge: Security and safety in the model context protocol ecosystem," 2025.

[2] N. Acharya and G. K. Gupta, "A formal security framework for MCP-based AI agents: Threat taxonomy, verification models, and defense mechanisms," 2026.

[3] C. Huang, X. Huang, N. P. Tran, and A. Milani Fard, "Model context protocol threat modeling and analyzing vulnerabilities to prompt injection with tool poisoning," 2026.

[4] M. Rostamzadeh, S. Narula, N. Birhan, M. Ghasemigol, and D. Takabi, "MCP-DPT: A defense-placement taxonomy and coverage analysis for model context protocol security," 2026.

[5] D. Zhang, Z. Li, X. Luo, X. Liu, P. Li, and W. Xu, "MCP security bench (MSB): Benchmarking attacks against model context protocol in LLM agents," 2025, iCLR 2026.

[6] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning attack on real-world MCP servers," 2025.

[7] A. Sanna, "AgentDefense-Bench," https://github.com/arunsanna/AgentDefense-Bench, 2026.

[8] Z. Wang, Q. Chang, H. Patel, S. Biju, C.-E. Wu, Q. Liu, A. Ding, A. Rezazadeh, A. Shah, Y. Bao, and E. Siow, "MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers," 2025.

[9] Model Context Protocol contributors, "Model context protocol specification, revision 2026-07-28," https://modelcontextprotocol.io/specification/2026-07-28, 2026, streamable HTTP transport (§Server Validation, §Value Encoding), `patterns/mrtr`, `server/utilities/caching`, Tasks extension, deprecated-features registry.

[10] M. Bhatt, V. S. Narajala, and I. Habler, "ETDI: Mitigating tool squatting and rug pull attacks in model context protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control," 2025.

[11] L. Mas, J. Vilaplana, J. Rius, R. Spaimoc, and J. Mateo, "The trustworthy model context protocol (MCP) registry: An architectural blueprint for cryptographic provenance and runtime integrity," *Future Internet*, vol. 18, no. 5, p. 243, 2026.

[12] Snyk, "agent-scan (formerly invariant labs MCP-Scan)," Vendor tooling, 2026.

[13] National Security Agency, AI Security Center, "MCP security: Security design considerations for AI-driven automation," NSA, Tech. Rep. Cybersecurity Information Sheet U/OO/6030316-26, 2026. [Online]. Available: https://www.nsa.gov/Portals/75/documents/Cybersecurity/CSI_MCP_SECURITY.pdf

[14] OWASP GenAI Security Project, "OWASP top 10 for LLM applications (2025)," 2025. [Online]. Available: https://genai.owasp.org/llm-top-10/

[15] ——, "OWASP top 10 for agentic applications (2026)," 2025. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[16] Model Context Protocol contributors, "SEP-1865: MCP apps," Model Context Protocol Specification Enhancement Proposal, Final 2026-01-26, 2026.

[17] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[18] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.